

第四章梳理

下面我把 **Chapter 4: Greedy Algorithms (贪心算法)** 系统梳理一遍。你给的笔记已经把主干抓得很准了；再结合你上传的这份 PDF，可以看出本章在文件中的实际顺序大致是：

- 4.1 区间调度 (interval scheduling) 与区间分区 (interval partitioning)
- 4.2 最小化最大延迟 (minimizing maximum lateness)
- 4.3 最优缓存 (optimal caching)
- 4.4 图的最短路径 (Dijkstra)
- 4.5 最小生成树 (MST)
- 4.6 Kruskal 的 Union-Find 实现
- 4.7 聚类 (clustering)
- 4.8 哈夫曼编码与数据压缩
- 4.9 最小开树形图 (minimum arborescence)

我会按“**贪心思想** → **各题型** → **证明方法** → **容易混淆点**”来讲，而且尽量把“为什么这个贪心对、别的为什么错、证明到底在证明什么”讲透。

让我们更形式化地说明算法。我们用 R 来表示尚未接受或拒绝的请求集，并用 A 来表示接受的请求集。有关算法运行方式的示例，参见图 4-2。

```
初始令  $R$  是所有请求的集合，令  $A$  为空集
While  $R$  不空
    选择一个请求  $i \in R$ ，它有最小结束时间
    将请求  $i$  加入  $A$ 
    从  $R$  中删除所有与  $i$  不相容的请求
Endwhile
Return 集合  $A$  作为接受的请求集合
```

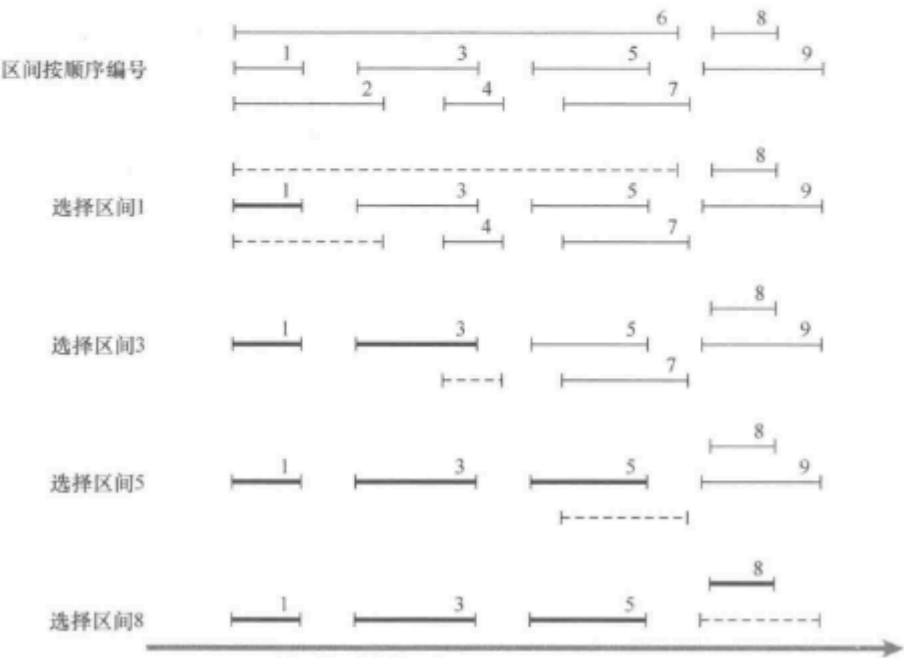


图 4-2 区间调度算法的运行例子。在每个步骤中，所选择的区间是较暗的线，在相应的步骤中删除的区间用虚线

```
按开始时间对区间排序，任意打破区间的并列
设  $I_1, I_2, \dots, I_n$  表示按这种顺序排列的区间
For  $j = 1, 2, 3, \dots, n$ 
    For 每个区间  $I_i$ ，它排在  $I_j$  之前且与  $I_j$  重叠
        考虑  $I_j$  时排除  $I_i$  的标签
    Endfor
    If 在  $\{1, 2, \dots, d_j\}$  中还有标签未被排除 then
        分配一个未排除的标签给  $I_j$ 
    Else
```

```
        保留  $I_j$  不分配标签
    Endif
Endfor
```

我们将按此顺序安排所有作业。同样，设 s 是所有作业的开始时间。作业 1 将在时刻 $s =$ 开始，并在时刻 $f(1) = s(1) + t_1$ 结束；作业 2 将在时刻 $s(2) = f(1)$ 开始，并在时刻 $f(2) = s(2) +$ 束；等等。我们将用 f 表示最后安排的作业的完成时间。下面是这个算法。

```
将作业按截止时间排序
为表示方便，假定  $d_1 \leq \dots \leq d_n$ 
初始化  $f = s$ 
按这个顺序考虑作业  $i = 1, \dots, n$ 
    将作业  $i$  分配给时间区间  $s(i) = f$  至  $f(i) = f + t_i$ 
    令  $f = f + t_i$ 
End
返回安排的区间集合  $[s(i), f(i)]$ ，其中  $i = 1, \dots, n$ 
```

```
Dijkstra 算法( $G, \ell$ )
令  $S$  是要探索的节点集
For 每个  $u \in S$ , 保存一个距离  $d(u)$ 
初始  $S = \{s\}$  且  $d(s) = 0$ 
While  $S \neq V$ 
    选择一个节点  $v \notin S$ ，使得从  $S$  到  $v$  至少有一条边，且
         $d'(v) = \min_{e=(u,v): u \in S} d(u) + \ell_e$  最小
    将  $v$  加入  $S$  并定义  $d(v) = d'(v)$ 
Endwhile
```

该算法的具体描述如下。

```
对给定频率的字母表  $S$  构建前缀码：
If  $S$  有两个字母 then
    将一个字母编码为 0，另一个字母编码为 1
Else
    令  $y^*$  和  $z^*$  是频率最低的两个字母
    通过删除  $y^*$  和  $z^*$ ，并用一个新字母  $w$  代替它们，其频率是  $f_{y^*} + f_{z^*}$ ，构造一个新字母表  $S'$ 
    递归地为  $S'$  构建前缀码  $\gamma'$ ，使用树  $T'$ 
    定义  $S$  的前缀码如下：
        从  $T'$  开始
        取标记为  $w$  的叶节点，并在它下面加上标记为  $y^*$  和  $z^*$  的两个子节点
Endif
```

我们称之为哈夫曼算法 (Huffman's Algorithm)，它为给定字母表产生的前缀码因此被称

一、先把整章的核心讲清：贪心算法到底在做什么

贪心算法不是“随便挑一个看起来最好的”。它的标准结构是：

1. 问题是一个全局优化问题；
2. 解是一点一点构造出来的；
3. 每一步都做一个当前看来最自然、最局部优的决定；
4. 决定一旦做出，就不回头修改。

所以贪心算法最难的从来不是“写伪代码”，而是两件事：

第一，局部规则到底该怎么选。

很多题都有好几个“看起来也很合理”的规则，但通常只有一个是真的。

第二，为什么这一步局部选择是安全的。

也就是：我现在贪了这一口，会不会把未来的全局最优破坏掉？

从这一章整体看，贪心正确性的证明大致反复出现三类语言：

- **stays ahead (保持领先)**：证明贪心在每一步都不比最优解落后；
- **exchange argument (交换论证)**：把任意最优解一步步改造成贪心解而不变差；
- **structural proof (结构下界)**：先找出任何解都绕不过去的下界，再证明贪心恰好达到它。

你可以把第四章理解成：

不是学几道题，而是在学这三种证明范式。

二、4.1 区间调度：为什么“最早结束优先”是对的

这一部分在 PDF 前几页，先讲区间调度，再顺带扩展到区间分区。

1. 问题定义

每个请求或作业 j 有开始时间 s_j 和结束时间 f_j 。

若两个区间不重叠，就称它们 **compatible (相容)**。

目标是：

从所有区间中选出一个**最大规模的两两相容子集**。

注意，这里不是安排所有任务，而是可以舍弃一些，只求“选得最多”。

2. 为什么这题很适合讲贪心

因为它天然就是“逐步挑区间”的题：

- 先挑一个；
- 删掉和它冲突的；
- 再从剩下的里挑一个；
- 如此继续。

难点不在“是不是一步步选”，而在：

3. 先看几个看起来合理、其实错误的规则

PDF 这一部分先故意列了几种直觉性规则，再逐个反驳。这个教学顺序很重要，因为它说明：

贪心的难点不是想出一个规则，而是知道哪个规则真的安全。

规则 A：最早开始时间优先

错因是：

一个区间虽然开始早，但可能结束得特别晚，一下子堵死后面很多短区间。
所以“开始早”不等于“占资源划算”。

规则 B：最短区间优先

错因是：

某个短区间可能刚好卡在中间，自己虽短，却同时挡住左右两边更好的组合。
所以“自己短”不等于“全局占位好”。

规则 C：冲突数最少优先

错因是：

“与别人冲突少”只是局部统计，不代表它在最优链条中的位置最关键。
一个区间冲突少，也可能恰好破坏最优组合的骨架。

所以这一题最值得学的第一个习惯就是：

面对贪心规则，先想反例。

4. 正确贪心：最早结束时间优先（earliest-finish-time-first）

正确算法是：

1. 按结束时间 f_j 从小到大排序；
2. 从前往后扫描；
3. 只要当前区间与已选集合兼容，就选入。

它之所以合理，不是因为“结束早听着顺”，而是因为它真正抓住了资源瓶颈：

选了一个区间以后，最希望它**尽快结束**，从而把未来时间尽量空出来。

这和“最短区间优先”不同：

- 最短只看长度；
- 最早结束同时看了它在时间轴上的位置。

所以 earliest finish 的本质是：

最小化当前一步对未来空间的占用。

5. 实现为什么是 $O(n \log n)$

先按结束时间排序，花 $O(n \log n)$ 。

然后线性扫描，每个区间只和“最后一个被选中的区间”比较一次，花 $O(n)$ 。

所以总复杂度是：

$O(n \log n)$

这也是很多贪心算法的经典模式：

排序 + 一次线性扫描。

6. 为什么它最优：stays ahead 证明

设贪心选出的区间依次是：

$i_1, i_2, \dots, i_k$

某个最优解选出的区间依次是：

$j_1, j_2, \dots, j_m$

证明关键是对每个 $r \leq k$ 证明：

$f(i_r) \leq f(j_r)$

也就是：

贪心选出的第 r 个区间，结束时间不晚于任何最优解的第 r 个区间。

这就叫 **greedy stays ahead**。

为什么成立？

- 对 $r = 1$ 很明显：贪心挑的是全体结束最早的。
- 若对 $r - 1$ 成立，则由于最优解中 j_r 必须在 j_{r-1} 结束后开始，而贪心有 $f(i_{r-1}) \leq f(j_{r-1})$ ，所以 j_r 在贪心做第 r 步时仍然是可选的。
- 既然 j_r 可选，贪心又总挑当前结束最早的，所以 $f(i_r) \leq f(j_r)$ 。

最后若最优解比贪心多一个区间，即 $m > k$ ，则由上式可推出 j_{k+1} 在贪心选完 i_k 后仍兼容，于是贪心本不该停止，矛盾。

所以 $k = m$ ，贪心最优。

这就是本章第一种最经典的证明模板。

三、4.1 扩展：区间分区（interval partitioning）为什么是另一道题

这部分紧接在区间调度之后。虽然输入形式还是一堆区间，但目标已经完全变了。

1. 问题定义

这里不再允许丢任务，而是：

必须安排所有区间，同时使用的资源数（如教室数）尽量少。

所以：

- 区间调度：选子集最大；
- 区间分区：全都要安排，资源最少。

这是两道不同的问题，千万不要混。

2. 先找下界：depth

定义 **depth**：

某一时刻同时覆盖该时刻的区间数的最大值。

如果某个时刻有 4 个区间同时进行，那么无论怎么排，这 4 个区间都不可能放进少于 4 个教室里。

所以：

任何方案所需资源数 $\geq \text{depth}$

这个量是天然下界。

3. 正确贪心：按开始时间排序，能复用就复用，否则新开

算法是：

1. 按开始时间递增排序；
2. 依次处理每个区间；
3. 若有某个已有资源当前已空闲，就放进去；
4. 若没有，就新开一个资源。

这题里“最早开始优先”反而是对的。为什么？

因为现在目标不是选得多，而是**尽量复用已有资源**。
按开始时间处理，刚好对应现实中的“事件到来顺序”。

4. 为什么要用优先队列

对每个教室，只需关心它当前最后一门课的结束时间。
把所有教室按“最早结束时间”放进优先队列。

处理新区间时：

- 先看最早空出来的教室；
- 若它都来不及接当前区间，那其他教室更不可能接；
- 若它可以接，就把当前区间放进去并更新结束时间。

所以这题用优先队列维护的是：

当前最早可复用的资源。

实现复杂度：

- 排序： $O(n \log n)$
- 每个区间一次堆操作： $O(\log n)$

总共仍是：

$O(n \log n)$

5. 为什么它最优：结构下界证明

这题不再用 stays ahead，而是用结构下界。

设贪心总共用了 d 个教室。考虑第 d 个教室被打开的那个时刻，对应某个区间 j 。

之所以必须新开，说明当时前 $d - 1$ 个教室都还没空出来，也就是有 $d - 1$ 个区间与 j 在该时刻重叠。于是某个时刻至少有 d 个区间同时进行。

所以：

$\text{depth} \geq d$

而任何方案都至少要 depth 个资源，所以任何方案都至少要 d 个资源。
贪心恰好用了 d 个，因此它最优。

这就是本章第二种证明语言：

先找谁都绕不过去的下界，再证明贪心达到它。

四、4.2 最小化最大延迟：为什么 EDF 是对的

这一部分在 PDF 中段开头，重点是交换论证。

1. 问题定义

每个作业 i 有：

- 处理时间 t_i
- 截止时间 d_i

调度后完成时间记为 f_i ，延迟定义为：

$$l_i = \max 0, f_i - d_i$$

目标不是最小化总延迟，而是最小化：

$$L = \max_i l_i$$

也就是：

最小化最大延迟。

这是一个 min-max 目标，和“总延迟最小”是两回事。

2. 先排除错误规则

最短处理时间优先（SPT）是错的

因为它完全不看 deadline。

可能先做一个很短但完全不着急的任务，反而把真正紧急的长任务拖迟。

最小 slack 优先也是错的

所谓 slack 是 $d_i - t_i$ 。

它看起来像“最危险先做”，但仍然只是局部量，不能保证全局最大延迟最小。

3. 正确贪心：最早截止时间优先（EDF）

算法就是把作业按 deadline 递增排序，然后从时间 0 开始无空闲地一个个做。

这个规则背后的直觉是：

若早截止任务被拖后，它更容易成为“最大延迟”的制造者。

所以 EDF 抓住的是“紧迫性顺序”。

4. 为什么可以先只考虑“无空闲时间”的调度

如果某个最优调度中间有空档，而后面还有作业没做，那把后面作业整体左移一点，只会让完成时间更早，不会更坏。

因此总存在一个最优调度：

- 没有 idle time；
- 只差一个作业顺序问题。

这一步很重要，因为它把问题从“连续时间安排”简化成了“排列作业顺序”。

5. 交换论证的核心：逆序（inversion）

先按 deadline 编号，使得：

$$d_1 \leq d_2 \leq \dots \leq d_n$$

若某调度里出现 j 在 i 前面，但 $d_i < d_j$ ，就称 (i, j) 是一个 **inversion**。

也就是说：

本该 deadline 更早的任务在前，你却把更晚的放到了前面。

EDF 恰好就是“没有 inversion 的调度”。

6. 为什么交换一对相邻逆序不会让最大延迟变坏

这是整道题的灵魂。

设相邻逆序是 j, i ，且 $d_i < d_j$ 。交换成 i, j 。

交换后：

- i 提前完成，所以它的延迟只会变小；
- j 虽然后移了，但它 deadline 更晚，可以证明它交换后的延迟不会超过交换前 i 的延迟。

因此，整个调度的最大延迟不会增加。

同时，逆序数减少了 1。

7. 所以 EDF 必然最优

从任意一个最优调度出发：

- 若它有逆序，就找一对相邻逆序交换；
- 交换后仍然最优，但逆序数更少；
- 反复进行，最终得到一个没有逆序的最优调度。

而没有逆序且无空闲的调度就是 EDF。

因此 EDF 最优。

这就是本章第三种核心证明语言：

| exchange argument（交换论证）。

五、4.3 最优缓存：更复杂的交换论证

这一段在 PDF 中紧接着 lateness。

1. 问题

缓存容量为 k ，请求序列为：

$d_1, d_2, \dots, d_m$

每次请求：

- 若命中缓存，叫 hit；
- 若不在缓存，叫 miss，需要装入；
- 若缓存已满，还得逐出一项。

目标是：

| 最小化总 miss 数。

2. 正确贪心：Farthest-in-Future (FF)

当必须逐出某项时，逐出：

未来最晚才会再次被请求的那个
(若以后再也不用，更应该逐出)。

这条规则很直观：

既然近期用不到，留着它不如留近期会用到的。

3. 为什么要先引入 reduced schedule

所谓 reduced schedule 是指：

只在一个元素真正被请求而且不在缓存时，才把它装入缓存。

也就是不做“无意义的提前预取”。

这样做的目的是缩小要分析的最优解范围：

任何非 reduced 调度，都可以改造成一个不更差的 reduced 调度。

4. 为什么 FF 最优：一步步把最优解改造成 FF

证明思想和 4.2 相同，但复杂得多。

设某个最优调度 S 在前 j 步与 FF 完全一致。

考虑第 $j+1$ 步第一次分歧：

- S 逐出的是 f
- FF 逐出的是 e

且 FF 之所以选 e ，正因为 e 的下一次请求比 f 更晚。

接下来要构造一个新调度 S' ：

- 在这一步模仿 FF，改成逐出 e ；
- 之后在未来某个时机把缓存状态重新接回 S ；
- 并保证总 miss 数不增加。

关键理由是：

因为 e 比 f 更晚才会用到，所以在真正再次用到 e 之前，总能先处理 f 相关的分歧，从而把状态修正回来。

于是可以把“与 FF 的一致前缀”从前 j 步扩展到前 $j+1$ 步。

反复做下去，最终把某个最优调度完全改造成 FF，且不变差，所以 FF 最优。

这一题告诉你：

交换论证不一定只是“交换相邻两个元素”，
也可以是“交换两种未来策略的行为，并在以后重新对齐”。

六、4.4 图的最短路径：为什么 Dijkstra 也是贪心

这一部分在 PDF 后半段开始。

1. 问题定义

给定带非负边权的有向图 $G = (V, E)$ 和源点 s ，要求每个点 v 的最短路距离：

$\text{dist}(s, v)$

路径代价等于边权和。

2. Dijkstra 的贪心选择是什么

维护一个已确定最短路的点集 S 。

对每个未探索点 v ，定义当前候选值：

$$\pi(v) = \min_{(u,v): u \in S} d(u) + \ell_{uv}$$

然后每一步贪心地选出 $\pi(v)$ 最小的那个点 v ，把它加入 S ，并宣告：

$$d(v) = \pi(v)$$

这就是“当前最短可确定点先定下来”。

3. 为什么这一步是安全的

证明关键是：

任取一条从 s 到 v 的任意路径 P ，看它**第一次离开集合 S** 的那条边 (x, y) 。

因为 $x \in S$ ，根据归纳假设，到 x 的最短距离已确定；

于是路径 P 的长度至少是：

$$d(x) + \ell_{xy} \geq \pi(y) \geq \pi(v)$$

所以任何到 v 的路径都不会短于 $\pi(v)$ 。

而我们已经有一条长度正好为 $\pi(v)$ 的候选路径，所以它就是真正最短路。

4. 非负边权为什么是必须的

这个论证依赖一个关键事实：

路径一旦离开 s 之后，后面不会靠负边把总长度“补得更短”。

如果允许负边，当前看起来最近的点，未来可能通过一条负边绕回来变得更短，贪心就失效。

所以 Dijkstra 只适用于：

边权非负。

5. 实现和复杂度

用优先队列维护所有未探索点的 $\pi(v)$ ：

- ExtractMin：取当前最小候选点
- ChangeKey：新加入点后更新邻居候选值

于是复杂度是：

$O(m \log n)$

如果用数组，也可以做到 $O(n^2)$ 。

这和 Prim 在实现上几乎平行。

七、4.5 最小生成树（MST）：为什么这里会有三种正确贪心

这一节是本章最“成体系”的部分，PDF 中也占了较大篇幅。

1. 问题定义

给定连通无向带权图 $G = (V, E)$ ，选边集 $T \subseteq E$ ，使得：

1. (V, T) 连通；
2. 无环；
3. 总权重最小。

这就是最小生成树（minimum spanning tree, MST）。

2. 为什么最优解一定是树

若一个最低成本解仍有环，那么删掉环上任意一条正权边，图仍连通，且总成本更低，矛盾。

所以最优解一定是：

- 连通
- 无环

也就是树。

3. 三种正确贪心算法

Kruskal

- 按边权从小到大排序；
- 若当前边不形成环，则加入。

它维护的是一个**森林**，不断把不同连通块拼起来。

Prim

- 从某个根开始；
- 每次加一条从当前树到外部的最便宜边。

它维护的是一棵**不断长大的树**。

Reverse-Delete

- 按边权从大到小考虑；
- 若删掉当前边后图仍连通，就删掉。

它维护的是一个**不断被修剪的连通图**。

课件和 PDF 都把这三种算法并列出来，强调它们都对。

4. 为什么三种都对：因为 MST 有两个全局结构性质

(1) Cut Property (割性质)

对任意 cut，跨越这个 cut 的最便宜边，一定属于每一棵 MST。

这说明某些边是“安全边”，可以放心加入。

(2) Cycle Property (环性质)

对任意环，环上最贵的边不属于任何 MST。

这说明某些边是“肯定冗余边”，可以放心排除。

5. Prim 与 Kruskal 为什么都能用 cut property 证明

Prim

当前树的顶点集 S 自然形成一个 cut。

Prim 每一步选的正是跨越 $(S, V - S)$ 的最便宜边，所以安全。

Kruskal

当前边连接两个不同连通分量，这两个分量之间也定义了一个 cut。

由于边是按权重从小到大扫描的，当前边就是该 cut 上最便宜可用边，所以也安全。

6. Reverse-Delete 为什么可用 cycle property 证明

若某条当前考虑的边位于某个环上，并且删掉后图仍连通，那么它就是这个环里可以被删除的“冗余最贵边”。

所以 Reverse-Delete 本质上是在不断利用：

环上最贵边不该留在 MST 中。

7. Prim 的实现为什么像 Dijkstra

Prim 维护的是每个外部点与当前树之间的最小连接代价：

$$a(v) = \min_{(u,v), u \in S} c_{uv}$$

然后每次取最小 $a(v)$ 的点加入。

形式上和 Dijkstra 维护 $\pi(v)$ 非常像，所以也可用优先队列实现：

$$O(m \log n)$$

八、4.6 Kruskal 的 Union-Find 实现：为什么并查集在这里是核心

这一部分在 PDF 中是专门独立的一节。

1. Kruskal 的关键操作是什么

当处理边 (u, v) 时，要快速判断：

u 和 v 当前是否已经在同一个连通分量中？

若是，则加边会成环；

若不是，则可加入并把两个分量合并。

这正是并查集（Union-Find）的标准用途。

2. Union-Find 支持哪些操作

- $\text{Find}(x)$ ：查元素属于哪个集合
- $\text{Union}(x, y)$ ：合并两个集合

在 Kruskal 里：

- 每个连通分量就是一个集合；
 - 初始每个点单独成集；
 - 加边后做 Union。
-

3. 为什么要优化

若朴素维护集合名，Find 也许快，但 Union 可能要改很多元素；
若用树实现，又会有树变得很高的问题。

所以引入两个经典优化：

- 按大小 / 按秩合并
- 路径压缩

这样多次操作的总时间几乎线性，通常写成：

$$O(m\alpha(n))$$

其中 α 是反 Ackermann 函数，增长极慢，几乎可视为常数。

因此 Kruskal 的总复杂度主要由排序决定：

$$O(m \log n)$$

九、4.7 聚类（clustering）：为什么 Kruskal 提前停就能做最大间隔聚类

这一部分在 PDF 中紧接 Union-Find 后。

1. 问题定义

把对象集划成 k 个非空簇。

定义聚类的 **spacing** 为：

不同簇之间任意两点距离的最小值。

目标是：

找到一个 k -clustering，使 spacing 最大。

这叫 **maximum-spacing clustering**。

2. 算法：Kruskal 提前停止

做法是：

1. 初始每个点自己成一个簇；
2. 按边权从小到大扫描；
3. 每当一条边连接两个不同簇，就合并；
4. 当簇数减到 k 时停止。

这就是“把 Kruskal 做到还剩 k 个连通分量为止”。

等价说法是：

先求一棵 MST，再删去其中最贵的 $k - 1$ 条边。

3. 为什么它最优

设算法停止时下一条本会加入的边长度为 d ，那么该聚类的 spacing 就是 d 。

若存在另一个 k -聚类 spacing 大于 d ，那么所有长度小于 d 的边两端都必须在同一簇中，否则它的 spacing 不可能超过 d 。

这样一来，Kruskal 当前得到的每个簇在那个聚类里也必须整体保留。

但下一条长度为 d 的边连接了这两个簇，若 spacing 还要大于 d ，则这条边两端也必须要在同一簇里，于是簇数会少于 k ，矛盾。

所以该算法得到的就是最大 spacing 的 k -聚类。

十、4.8 哈夫曼编码与数据压缩：贪心如何构造最优前缀码

这一部分在 PDF 后面占了很大篇幅。

1. 压缩的基本目标

若每个字符都用同样长度编码，就是定长编码。
但现实中字符频率差异很大，所以更自然的是：

高频字符用短码，低频字符用长码。

这就是变长编码。

2. 为什么必须要求前缀码 (prefix code)

若一个字符编码是另一个编码的前缀，就会导致解码歧义。
所以要求：

任意字符编码都不是另一个字符编码的前缀。

这类编码可以用一棵二叉树表示：

- 左边记 0，右边记 1；
- 每个字符放在叶子上；
- 根到叶子的路径就是编码。

于是最优编码问题变成：

构造一棵二叉树，使加权平均编码长度最小。

3. 目标函数

设字符频率为 f_x ，编码长度为 $|\gamma(x)|$ ，则平均编码长度是：

$$ABL(\gamma) = \sum_x f_x |\gamma(x)|$$

若用树表示，则就是：

$$ABL(T) = \sum_x f_x \cdot \text{depth}_T(x)$$

目标是让它最小。

4. 哈夫曼算法的贪心规则

每一步都取**频率最低**的两个字符 / 子树，把它们合并成兄弟节点，父节点频率为两者之和，再把新节点放回优先队列。

不断重复，直到只剩一棵树。

5. 为什么这个贪心是对的

关键结构结论是：

在某棵最优前缀码树中，频率最低的两个字符一定可以放在最深层，并且互为兄弟。

既然如此，就可以先把它们合并成一个新字符，频率为两者之和，把原问题缩成更小问题。

若缩小后的问题最优解决了，再把该新字符展开成原来两个字符，就得到原问题最优解。

所以哈夫曼算法的正确性依赖于：

- 贪心选择性质；
 - 最优子结构；
 - 归纳。
-

6. 实现复杂度

用最小堆维护当前频率最小的树：

- 每次取出两个最小；
- 合并后插回；

共做 $n - 1$ 次合并，所以总复杂度：

$O(n \log n)$

这里 n 是字符种类数。

十一、4.9 最小开树形图 (minimum arborescence)：为什么要“收缩环”

这一部分是 PDF 最后一节，也是本章最复杂的一节。

1. 问题定义

给定有向图 $G = (V, E)$ 和根节点 r ，要找一边集 F ，使得：

1. 从根 r 可以沿有向边到达所有其他节点；
2. 每个非根节点恰好有一条入边；
3. 总边权最小。

这叫 **minimum arborescence**，可以看成有向图版本的“最小生成树”。

2. 一个自然的贪心为什么不够

最自然的想法是：

对每个非根节点，选进入它的最便宜边。

如果这些边没有形成有向环，那它们就已经是最优解。

因为任何可行解都必须给每个非根点选一条入边，而你已经逐点选了最便宜的。

但问题是：这些边可能形成有向环。

这和无向图中的 MST 不同：

无向里成环可以靠 cut / cycle property 简单处理；

有向里由于方向约束，事情复杂得多。

3. 修正方法：收缩有向环（contract cycle）

若选出的最便宜入边形成了一个有向环 C ，就把整个环收缩成一个超级节点。

此时：

- 环内大多数贪心选择其实仍应保留；
- 真正待决定的是“环外哪条边进入这个环”。

为保持等价性，需要对进入超级节点的边做权重调整：

如果边 (u, v) 从环外进入环中节点 v ，则新图中的对应边成本改成：

$$c'(u, C) = c(u, v) - c(e_v)$$

其中 e_v 是原来给 v 选中的最便宜入边。

这个减法的意思是：

如果最终选择外部边进入 v ，那原来进入 v 的那条环内最便宜入边就要被删掉，所以真正新增的成本只是“替换差额”。

4. 递归求解与展开

于是算法变成：

1. 给每个非根点选最便宜入边；
2. 若无环，结束；
3. 若有环，就收缩该环并调整边权；
4. 在收缩后的图中递归求最小开树形图；
5. 最后再把超级节点展开，并恢复原图中的边。

这是一种**多阶段贪心 + 缩约递归**的结构。

5. 这一题最值得学的点

它告诉你：

有时“最自然的局部贪心”并不直接可行，但并不意味着贪心思想错了。
你可能需要先接受局部结构，再通过压缩冲突结构（这里是环）把问题递归化。

所以这一节其实是在把贪心范式推进到更复杂的层次。

十二、你笔记里额外补的两个课件扩展：找零与最少加油停靠

你给的笔记里还额外补了课件中的两个扩展例子，这两个很值得一并纳入第四章的理解框架。

1. 找零问题 (coin change)

- 对某些币制，如美元常见币制，贪心“每次拿不超过剩余金额的最大面值”是对的；
- 但对一般币制并不成立，例如你前面提到的 1, 10, 21, 34, 70, 100, ... 凑 140 的反例。

这题最重要的结论不是“贪心总能找零”，而是：

贪心正确性依赖于币制结构。

也就是 canonical coin system 的思想。

2. 最少加油停靠 (minimum refueling stops)

正确贪心是：

每次都尽量开到当前油量能达到的最远断点。

为什么对？因为它能用“最长公共前缀 + 交换论证”来证明：

任一最优解若在某一步没像贪心一样走到最远，都可以把那一步替换成贪心走法而不变差。

所以这题本质上和区间调度、EDF 属于同一交换论证家族。

十三、把整章真正串起来：Chapter 4 到底想让你学会什么

如果把整章压成一条主线，其实就是这四句话。

1. 同一个“看起来合理”的规则，换个目标就可能错

- 区间调度里 earliest start 是错的；
- 区间分区里 earliest start 却是对的；
- 找零在某些币制里对，在另一些币制里错。

所以：

| 贪心规则永远依赖于**目标函数**和**问题结构**。

2. 贪心不是拍脑袋，而是要找“安全的局部选择”

- interval scheduling：最早结束是安全的
- interval partitioning：最早开始 + 资源复用是安全的
- Dijkstra：当前最短可确定点是安全的
- MST：cut 上最轻边、cycle 上最重边分别给出安全加边和安全删边
- Huffman：最低频字符符合并是安全的

3. 正确性证明有几种固定范式

这一章基本覆盖了三大典型模式：

- **stays ahead**：区间调度
- **exchange argument**：最大延迟、缓存、加油停靠
- **structural proof / lower bound**：区间分区、聚类
- 再往后还有：cut/cycle property、归纳、收缩递归

4. 数据结构往往决定贪心能否真的高效落地

- interval partitioning：priority queue
- Dijkstra / Prim：priority queue
- Kruskal：Union-Find
- Huffman：min-heap

所以：

| 算法思想给你“方向”，数据结构给你“效率”。

十四、最后给你一份“第四章最该掌握的知识骨架”

必须准确区分的题型

- 区间调度：选最多个
- 区间分区：全都安排、资源最少
- 最大延迟最小化：最坏延迟最小，不是总延迟最小
- 缓存：最少 miss
- 最短路径：源点到各点距离最小
- MST：总边权和最小
- 聚类：spacing 最大
- Huffman：平均编码长度最小
- 最小开树形图：有向根树成本最小

必须记住的正确贪心规则

- interval scheduling：earliest finish time
- interval partitioning：earliest start time + 最早空闲资源复用
- minimizing lateness：earliest deadline first
- optimal caching：farthest in future
- Dijkstra：当前最小 tentative distance
- Prim：跨当前树最轻边
- Kruskal：不成环的最轻边
- Huffman：合并频率最低的两个节点
- refueling：每次到当前能到达的最远站

必须会说的证明语言

- stays ahead
- exchange argument
- structural lower bound
- cut property
- cycle property
- greedy choice + optimal substructure
- contraction / recursive repair

必须会说的关键复杂度

- interval scheduling： $O(n \log n)$
- interval partitioning： $O(n \log n)$
- Dijkstra（堆）： $O(m \log n)$
- Prim（堆）： $O(m \log n)$

- Kruskal: $O(m \log n)$
 - Huffman: $O(n \log n)$
-

如果你愿意，我下一条可以继续把 **Chapter 4 压成一版“考前复习提纲”**，按“题型—错误贪心—正确贪心—证明套路—复杂度—易错点”做成一份更方便背诵的版本。